

Tema 1: Fundamentos de diseño de computadores. Sistemas multinúcleo

Introducción

En este tema se aborda una primera aproximación a los procesadores multinúcleo, estudiando su estructura general, su fabricación y modelos simples para entender su funcionamiento. Se estudiará por qué el modelo seguido hasta hace pocos años no era escalable y cuáles han sido los cambios que se han hecho para continuar la mejora de las prestaciones.

Arquitectura de un computador

En el pasado se concebía la arquitectura de un computador como el ISA, es conjunto de instrucciones de la arquitectura. Esto incluye los registros, la forma de acceso a la memoria, operaciones disponibles, etc.

Hoy en día la arquitectura engloba más cosas, como la arquitectura de los microchips y el hardware. La arquitectura se diseña para satisfacer los requisitos específicos de las máquinas y maximizar el rendimiento dentro de ciertas restricciones (coste, potencia y disponibilidad).

Tipos de paralelismo

El paralelismo es el mecanismo más utilizado recientemente para tratar de mejorar el rendimiento de las máquinas. Existen varios tipos de paralelismo.

Paralelismo en las aplicaciones:

- Paralelismo a nivel de datos - *Acceso a múltiples datos a la vez*
- Paralelismo a nivel de tareas - *Múltiples tareas simultáneas*

Paralelismo en las arquitecturas

- Paralelismo a nivel de instrucciones - *Múltiples instrucciones ejecutadas simultáneamente ya agotado*
- Arquitecturas vectoriales, GPUs
- Paralelismo a nivel de hilo
- Paralelismo a nivel de peticiones

Para explotar el paralelismo es necesario modificar las aplicaciones, haciendo explícito el uso de paralelismo.

Taxonomía de Flynn

Clasificación de las arquitecturas según sus flujos de datos e instrucciones:

- Single instruction stream, single data stream **SISD**
 - Computadores secuenciales que pueden explotar el paralelismo a nivel de instrucción
- Single instruction stream, multiple data streams **SIMD**
 - Una misma instrucción se aplica sobre distintos flujos de datos. Arquitecturas vectoriales, GPUs o extensiones SSE.

- Multiple instruction streams, single data stream **MISD**
 - Sin mucho sentido.
- Multiple instruction streams, multiple data streams **MIMD**
 - Múltiples flujos de instrucciones actúan sobre conjuntos de datos.

Las arquitecturas MIMD son las más comunes

Arquitecturas MIMD

Existen dos tipos principales de arquitecturas MIMD: aquellas con memoria compartida y las de memoria distribuida.

- Memoria compartida: todos los procesadores comparten el mismo espacio de direcciones y el programador no necesita saber la ubicación de los datos
- memoria distribuida: cada procesador tiene su espacio de memoria.

Según la latencia en el acceso a memoria se puede realizar la clasificación:

- Acceso uniforme a memoria UMA: latencia independiente del procesador, máximo 128 procesadores
- Acceso no uniforme NUMA: latencia dependiente del procesador, más baja para memoria local.
 - NCC-NUMA - Sin coherencia cache
 - CC-NUMA - Con coherencia cache

Arquitecturas mononúcleo

Su apogeo se da en 2003 con el Pentium 4. Incluye hardware para la ejecución fuera de orden y unidades de ejecución en paralelo

Arquitecturas multinúcleo

Múltiples núcleos en un único procesador. Estos comparten ciertos recursos como caché L3 o procesador gráfico.

Pipelines

Ejecución de las instrucciones por etapas separadas, de forma que etapas de múltiples instrucciones se puedan ejecutar simultáneamente. Entre las etapas se encuentran registros que provocan un retardo, habitualmente medido en F04, que es el retardo equivalente a un inversor conectado a cuatro inversores de salida.

Evolución de la tecnología

A lo largo de los años la tecnología de fabricación de los procesadores ha evolucionado, consiguiendo cada vez transistores más pequeños. El **nodo tecnológico** es el valor numérico que indica el ancho mínimo de una conexión de cobre de nivel 1. A lo largo de los años se ha reducido el nodo tecnológico como consecuencia de la mejora en la fabricación de los microprocesadores.

Relaciones numéricas

Con la evolución de los procesadores se han observado ciertas relaciones empíricas entre características como la energía y potencia dinámicas, potencia estática, etc. Algunas son:

- En. din. = $\frac{1}{2} \times \text{Carga Capacitiva} \times \text{Voltaje}^2$
- Pot. din. = $\frac{1}{2} \times \text{Carga Capacitiva} \times \text{Voltaje}^2 \times \text{Frecuencia}$
- Pot. est. = Corriente estática fugas $\times$ Voltaje

Se pueden estudiar técnicas para reducir el consumo de energía como por ejemplo reducir el voltaje dinámicamente.

También existen relaciones con el ancho de banda y la latencia:

- Ancho banda máx. = $(N \text{ núcleos} \times \text{Frecuencia}) / (\text{Intensidad operacional} \times \text{CPI del núcleo})$
- Latencia³ $\times$ Potencia = cte(Nodo tecnológico)

Existen muchas otras relaciones utilizadas para deducir la manera de mejorar ciertas prestaciones en los procesadores.

Agotamiento del escalamiento mononúcleo

El modelo basado en un solo núcleo se ha agotado (aproximadamente en 2005). Esto es debido a que todas las mejoras posibles con un solo núcleo se han optimizado, y existen barreras físicas que limitan el progreso (longitud de conexiones, por ejemplo). Tras este agotamiento fue necesario explorar nuevas opciones, principalmente los procesadores multinúcleo.

Métricas de rendimiento

Para medir la mejora de rendimiento en los programas se utilizan ciertas métricas habituales

- Tiempo de respuesta: $T_{\text{inicial}} - T_{\text{final}}$
- Throughput: Trabajo por unidad de tiempo
- Speedup de X respecto a Y: T_y / T_x
- Tiempo de CPU: solo incluye el tiempo que se utiliza la CPU, sin sobre costes del SO.
- CPI, media de ciclos por instrucción: $(\text{Suma CPI}_i \times N_i) / N$

Tema 2: Núcleos de procesamiento.

Segmentación

Arquitectura MIPS

Ya ha sido vista en Fundamentos de Computadores, es una arquitectura tipo RISC. En la asignatura previa se vio la implementación monociclo, pero ahora veremos la versión multiciclo.

Multiciclo y segmentación

Con la división de la ejecución de las instrucciones en varias etapas se aprovechan mejor los tiempos variables de ejecución de las instrucciones. Se realiza *pipelining*.

Segmentación del cauce

Para segmentar la ejecución de las instrucciones se necesita hardware adicional como registros intermedios para evitar interferencia entre las fases. Además, se deben investigar los problemas que surgen de la nueva forma de ejecutar las instrucciones, específicamente las dependencias entre ellas.

El cauce de ejecución se divide varias fases. Se tiene un primer planteamiento de 5 fases:

- **IF:** búsqueda de las instrucciones en memoria.
- **ID:** decodificación de las instrucciones y lectura del banco de registros
- **EX:** ejecución o cálculo de direcciones. Tiene latencia variable dependiendo del tipo de instrucción.
- **MEM:** acceso a memoria para acceso a un dato.
- **WB:** escritura de los resultados.

Es importante denotar que la aceleración obtenida con segmentación se obtiene por aumento del *throughput*, pero no reducción de la latencia.

Ejemplo

Instrucción `lw`

- **IF:** obtención de la instrucción desde memoria
- **ID:** lectura del registro base de la dirección y extensión del signo del inmediato (offset).
- **EX:** suma de la base y el offset.
- **MEM:** lectura de la memoria (caché) utilizando la dirección recién calculada
- **WB:** almacenamiento del valor leído en el banco de registros, cambiando hardware para evitar interferir.

Instrucción `sw`

- **IF:** obtención de la instrucción desde memoria
- **ID:** lectura del registro base de la dirección, del registro a almacenar y extensión del signo del inmediato (offset).
- **EX:** suma de la base y el offset.
- **MEM:** escritura de la memoria (caché) utilizando la dirección recién calculada y el contenido del registro.
- **WB:** no se hace nada.

Riesgos en la ejecución

Al realizar paralelismo a nivel de instrucción aparecen *riesgos de ejecución*, que son problemas que surgen por la ejecución simultánea de instrucciones. Existen varios tipos:

- **De estructura:** el hardware no permite la ejecución de ciertas instrucciones simultáneamente.
- **De datos:** hay necesidad de esperar a que una instrucción anterior lea o escriba el datos antes de leerlo o escribirlos nosotros.
- **De control:** surgen del problema de detectar que instrucción sigue a una instrucción de salto.

Riesgos de estructura

Si no se dispone del hardware necesario pueden existir instrucciones incompatibles. Por ejemplo, si solo se tiene una memoria compartida de datos e instrucciones, no se

pueden ejecutar en el mismo ciclo las fases **IF** y **MEM**. Para solucionar el problema se modificar el hardware, introduciendo nuevas unidades funcionales.

Riesgos de datos

Habitualmente las instrucciones dependen de datos que utilizan o generan otras instrucciones, previas o posteriores. Esto puede producir riesgos

Tipo RAW

Read after write o dependencia verdadera. Se producen cuando la instrucción **j** intenta leer un dato antes de que la instrucción **i** lo escriba, con **j** posterior a **i**.

Existen distintas estrategias para gestionarlos.

Tipo WAR

Write after read o antidependencia. Se producen cuando la instrucción **j** intenta escribir un dato antes de que la instrucción **i** lo lea, con **j** posterior a **i**.

En el MIPS con pipeline de 5 etapas no pueden ocurrir por el orden de la escritura y lectura.

Se soluciona con el renombrado de registros.

Tipo WAW

Write after write o falsa. Se producen cuando la instrucción **j** intenta escribir un dato antes de que la instrucción **i** lo escriba, con **j** posterior a **i**. Se produciría una alteración de la salida real.

En el MIPS con pipeline de 5 etapas no pueden ocurrir.

Se soluciona con el renombrado de registros.

Tratamiento de riesgos de datos

Existen distintas maneras de gestionar los riesgos de datos. Por software, el compilador puede reordenar ciertas instrucciones independientes para eliminar los riesgos o reducir su penalización. Por hardware, se pueden crear caminos de datos que adelanten los datos a otras instrucciones. Esto se denomina forwarding. Por último se puede detectar que existen riesgos, bloqueando las instrucciones hasta que se pueda avanzar.

Existen mecanismos más avanzados para detectar el forwarding y el interbloqueo.

Riesgos de control

Los riesgos de control aparecen cuando se ejecuta una instrucción de salto condicional. A priori no se puede saber cual será la siguiente instrucción a ejecutar. Existen varias soluciones

Parada

Se puede frenar la ejecución de instrucciones hasta que se resuelva el resultado del salto. Cuando más profundo sea el pipeline, más retardo se generará.

Decisión retardada

El compilador mueve varias instrucciones previas al salto a después de este. Estas instrucciones deben ser independientes del salto. De esta manera se aprovechan los ciclos en los que se decide el salto.

Predicción de salto

Existen dos tipos, predicción estática o dinámica. La estática analiza el comportamiento típico de los saltos (p. ej. predecir que los saltos hacia atrás están asociados a bucles y se producirán la mayoría de las veces). La dinámica mantiene una forma de historial durante la ejecución real del salto, y predice el comportamiento en base a este historial.

Excepciones

Las instrucciones ejecutadas en el procesador pueden generar excepciones: código de instrucción indefinido, división entre 0, etc. Su gestión se dificulta cuando se segmenta el pipeline. Se presentan distintas soluciones, como por ejemplo incorporar bancos de registros adicionales o parar la emisión de instrucciones si se detecta riesgo de excepción.

Conclusiones

- El hardware del camino de datos y la lógica de control se complican con la segmentación
 - Caminos de anticipación *forwarding*
 - Lógica para detección de conflictos de datos
 - Lógica de predicción de saltos
- La segmentación mejora la productividad pero no reduce el tiempo de ejecución
- Incrementar la longitud del pipeline no tiene por qué mejorar el rendimiento
 - Los conflictos de datos hacen que al incrementar la longitud se incremente el tiempo invertido en cada instrucción
 - Los conflictos de control provocan que un incremento en la longitud ralentice los saltos.
 - La sobrecarga por los registros de segmentación puede limitar el aumento de la frecuencia de reloj

Tema 3: Núcleos de procesamiento.

Paralelismo a nivel de instrucción.

El paralelismo a nivel de instrucción *ILP* consiste en la ejecución simultánea de múltiples instrucciones. Se puede conseguir con la segmentación (pipeline) o con emisión múltiple, que consiste en replicar etapas del pipeline para emitir múltiples instrucciones en cada ciclo de reloj.

Dependencias de nombre

Las dependencias generan riesgos que limitan el paralelismo. En el tema anterior se analizaron las dependencias RAW. En este tema se verá que las dependencias WAR y WAW no son dependencias reales, son lo que se denomina dependencias de nombre. Esto quiere decir que no existen dependencias entre los datos producidos por las instrucciones,

solo entre el nombre de los registros. Suponen un problema en el reordenamiento de las instrucciones pero se solucionan con renombramiento de registros.

Emisión múltiple

Consiste en la replica de etapas del pipeline. Existen dos tipos:

Emisión múltiple estática

Es realizada por el compilador, que agrupa las instrucciones para emitirlas juntas en *issue packets*. El compilador también es responsable de detectar y evitar los riesgos. Solo es eficaz en aplicaciones científicas, no de propósito general.

Los *issue packets* contienen varias instrucciones que pueden ser emitidas en un solo ciclo, por estar disponible el pipeline y ser independientes las instrucciones. Se puede ver como una instrucción muy larga.

El compilador es responsable de eliminar los riesgos, reordenando y agrupando instrucciones sin dependencias. Posiblemente haya dependencias entre paquetes y en ocasiones sea necesario introducir instrucciones *nop*.

En la emisión múltiple estática surgen nuevos riesgos, específicamente entre instrucciones de un mismo paquete. No se podrá hacer forwarding en ciertas ocasiones, entre instrucciones del mismo paquete. El riesgo se denomina de carga-uso, cuando se carga un dato que se usa en otra instrucción (del mismo paquete).

Para planificar las instrucciones se plantean arquitecturas VLIW, con instrucciones que engloban varias operaciones (accesos a memoria independientes, varias operaciones con enteros, etc).

Emisión múltiple dinámica

La CPU es la responsable de examinar el flujo de instrucciones y seleccionar cuales emitir en cada ciclo. El compilador también puede ayudar, reordenando las instrucciones. La CPU resuelve los riesgos con técnicas avanzadas.

Las CPUs capaces de realizar estas tareas se denominan **superescalares**. Al hacerse cargo del trabajo la CPU se aumenta la complejidad pero se independiza al compilador del hardware específico.

Se permite a la CPU ejecutar instrucciones fuera de orden para evitar paradas, pero al final se escriben los resultados en orden para garantizar su consistencia.

La planificación dinámica es la labor que lleva a cabo la CPU para tratar de obtener el orden de instrucciones óptimo. Implica ejecución y terminación fuera de orden. Una ventaja añadida es que permite planificar respecto a condiciones que el compilador no puede conocer: fallos caché, resultados de saltos, latencias variantes por ISAs, etc.

Renombrado de registros

Para conseguir reordenar las instrucciones y evitar riesgos WAW y WAR se realiza renombrado de registros. Para ello se utilizan **estaciones de reserva**.

Estaciones de reserva: contenedores que almacenan una instrucción a ejecutar, los valores de los operandos (si están disponibles) y la validez de cada uno de ellos y el

número de estación de reserva de que proporcionará los valores de un cierto operando (si aún no está disponible).

Cuando una instrucción se emite a una estación de reserva:

- Si el operando está disponible en el almacén de registros o en un buffer de reordenamiento, se copia a la estación de reserva.
- Si no, se proporcionará en un futuro, puede que no sea necesario actualizar el propio registro.

Las instrucciones entran por orden a las RS, pero puede que se ejecuten fuera de orden, dependiendo de la disponibilidad de los operandos.

Cuando una instrucción finaliza, puede que envíe su resultado a otra RS que lo necesite y que vuelva a actualizar el registro. De esta forma solo se almacenará en el banco de registros el último valor calculado. A medida que se emiten instrucciones, los especificadores de registro se renombran con los nombres de las estaciones de reserva.

Algoritmo de Tomasulo

Este algoritmo se utiliza para gestionar el uso de las estaciones de reserva y buffers de reordenamiento. Consiste en tres pasos:

1. Emitir
 1. Se obtiene la siguiente instrucción de la cola FIFO.
 2. Si la RS asociada a la instrucción está disponible, emite la instrucción a la RS con los valores del operando, si estos están disponibles.
 3. Si no lo están, se detiene la instrucción.
2. Ejecutar
 1. Cuando el operando está disponible, lo almacena en cualquier estación de reserva que esté esperando por él.
 2. Cuando todos los operandos están listos, se emite la instrucción.
 3. Las cargas y el almacenamiento se mantienen en el orden del programa por la dirección efectiva.
 4. No se permite iniciar la ejecución de ninguna instrucción hasta que se hayan completado las ramas que la preceden en el orden del programa.
3. Escribir el resultado
 1. Se escribe el resultado en las estaciones de reserva y en los buffers de almacenamiento.

Especulación

La especulación es un mecanismo de intuición sobre lo que hará una instrucción antes de que se ejecute, para intentar ahorrar ciclos de ejecución. Se comienza una operación lo antes posible. Si se acertó se continúa, si no se descarta el resultado y se hace lo correcto.

Ejemplos de especulación:

- Predicción del salto y continuar emitiendo instrucciones. No se hace commit del resultado hasta que se verifica que es el correcto.
- Especulación en carga. Trata de cargar los datos evitando el retardo por carga y fallos caché.

Tipos de especulación

- Especulación del compilador: reordenamiento de instrucciones, puede incluir instrucciones de arreglo de especulación incorrecta.
- Especulación por hardware: el procesador busca por adelantado que instrucciones ejecutar, almacena los resultados en un buffer hasta conocer si son correctos. Si no lo son, se vacía el buffer.

Buffers de reordenamiento

Los buffers de reordenamiento son el mecanismo principal utilizado en la especulación basada en hardware. En los buffers de reordenamiento se almacena el resultado de la instrucción entre que se completa y se hace el commit. En cada entrada del buffer se almacenan distintos datos que permiten la utilización correcta del resultado.

Al utilizarlos, se deben modificar las RS, ya que sus resultados vendrán ahora de los buffers de reordenamiento.

Una cierta instrucción puede estar en varios estados:

- En ejecución: la instrucción no ha acabado de ejecutarse
- Finalizada: su resultado se escribe en un registro renombrado o en un buffer de escritura a memoria.
- Completada: cuando la instrucción modifica el banco de registros
- Retirada: cuando se escribe en la cache de datos (si es necesario).

En general, las etapas que se distinguen son:

- Emisión: asignar RS y ROB, leer los operandos disponibles.
- Ejecución: comenzar la ejecución cuando los operandos están disponibles.
- Escritura del resultado: escribir el resultado y la etiqueta del ROB en el buffer CDB.
- Commit de la instrucción: cuando el ROB alcanza su cabecera, actualizar el registro. Cuando un salto con predicción errónea alcanza la cabecera del ROB, descartar todas las entradas

Desventajas de la especulación

La especulación no es una solución perfecta. Al especular erróneamente se incurre en gastos adicionales energéticos y temporales. Además se aumenta la complejidad en gran medida.

Resumen: emisión múltiple

La emisión múltiple es una solución que funciona, pero no tanto como nos podría gustar. Existen muchos factores que dificultan la ejecución paralela de las instrucciones, es difícil encontrar paralelismo.

Falacias

- Es falso que procesadores con CPIs más bajos o frecuencia más rápida sean más rápidos en general.
- Los diseños más complejos pueden ser menos eficaces que diseños más simples de mayor tamaño. También al revés.
- No por tener más disponibilidad de recursos se van a poder aumentar el paralelismo a nivel de instrucción. Existe un límite en la mayoría de los

programas.

Conclusiones

- La ISA influencia el diseño del camino de datos
- El pipelining mejora la productividad en número de instrucciones finalizadas por unidad de tiempo. La latencia de cada instrucción no cambia.
- Riesgos: estructurales, de datos y de control
- Emisión múltiple y planificación dinámica: las dependencias limitan la cantidad de paralelismo y la complejidad del hardware se limita por el coste energético.

Tema 4: Coherencia caché

Introducción

Al utilizar memorias caché independientes en múltiples procesadores surgen problemas de coherencia y consistencia. Si un dato se modifica en una caché, pero su valor no se actualiza en otra pueden generarse problemas:

Coherencia: un sistema es coherente si cualquier lectura de un dato garantiza que se lee el valor más reciente de este.

En sistemas de un procesador el problema se presenta entre la MP y la caché. En multiprocesadores también aparece el problema entre cachés de distintos procesadores.

Condiciones para la coherencia

Para que un sistema sea coherente se deben cumplir ciertas condiciones:

- **Preservación del orden del programa:** si un procesador P escribe un dato en X y luego ese mismo procesador lo lee, sin escrituras de otro procesador, entonces debe ser el dato escrito previamente por P.
- **Vista coherente de la memoria:** si un procesador Q escribe en la posición X y otro procesador P lee de esa posición, debe leer el valor escrito por Q.
- **Serialización de escrituras:** dos escrituras a la misma posición X por dos procesadores cualesquiera deben ser vistas en el mismo orden por todos los procesadores.

Métodos utilizados en MP

Para el problema de coherencia producido entre MP y cachés existen distintas soluciones (ya vistas en FC):

- **Escritura directa:** cuando se modifica algo en la caché, se modifica directamente en la MP.
- **Post-escritura:** cuando se modifica algún dato, se almacena únicamente en la caché. Cuando el dato se saque de la caché, se escribe a MP.

Posibles soluciones

Se ve que no es posible solucionar la falta de coherencia entre cachés sin utilizar hardware específico. Se añadirá nuevo hardware que utilizarán los llamados *protocolos de coherencia caché*, que son responsables de garantizar la coherencia.

Se basarán en dos conceptos:

- **Escritura-actualización:** cuando se escribe, se comprueba si el dato es compartido con otras cachés. Si lo es, actualiza el dato en las otras cachés. Más costosa pero funciona a nivel de palabra.
- **Escritura-invalidación:** cuando se escribe, se distribuye una señal de invalidación que indica al resto de las cachés que el bloque ya no es válido. Más simple pero puede invalidar líneas innecesariamente si contienen múltiples palabras.

Respecto a estos conceptos, se diseñan dos tipos de protocolos de coherencia caché:

- **Snooping:** se basan en espiar constantemente el bus de datos para comprobar qué hacen las otras cachés y así mantener coherente la caché de cada procesador. No escalable por la cantidad de tráfico en el bus.
- **Basados en directorio:** se basan en un directorio común que almacena el estado de las líneas caché, no necesita un bus común. Es un planteamiento centralizado pero puede alcanzarse un cierto grado de distribución.

Protocolos basados en snooping

La invalidación es una estrategia que garantiza que un procesador tiene acceso exclusivo a un bloque antes de realizar una escritura. En snooping se usa el bus de memoria compartido para la invalidación, el cual espían todos los procesadores. No puede haber escrituras simultáneas ya que el bus se utiliza de manera exclusiva, lo cual serializa las escrituras.

Protocolo de invalidación en tres estados (MSI)

Todas las líneas caché del procesador tienen asociado un cierto estado de, en este caso, tres posibles:

- **Modificado (M):** este es el único componente que tiene una copia válida del bloque, el resto de cachés y la memoria tienen una copia no actualizada.
- **Compartido (S):** este bloque también está presente en otras cachés, y todas las copias están actualizadas.
- **Invalido (I):** este bloque no está físicamente en la caché o su valor ha sido invalidado por haberse producido una modificación en otro lugar.

Cuando el procesador accede a las líneas de la caché, ya sea para leer o escribir, suceden distintas situaciones.

- El procesador quiere leer **PrLeer** pero el dato no está en la caché. Se solicita con una petición **PtLec** que será respondida por la memoria o por otras cachés. La línea adquirida será compartida.
- El procesador quiere leer **PrLeer** y el dato ya está en su caché en estado no inválido (modificado o compartido). Entonces simplemente lo lee y no envía ninguna petición.
- El procesador quiere escribir **PrEsc** y el bloque no está en la caché o está en estado compartido. Entonces se emite una petición **PtLecEx** que reciben los demás procesadores que están espiando. Aquellos que tengan una copia, la invalidarán (incluida la MP).
- El procesador quiere escribir **PrEsc** y el bloque está en estado modificado en la caché. El procesador escribe sin enviar ninguna petición.
- Si el procesador espiando el bus detecta una petición **PtLecEx** de un bloque que está compartido en su caché, debe invalidarlo.

- Si el procesador espiando el bus detecta una petición **PtLec** o **PtLecEx** de un bloque que está modificado en su caché, debe enviarlo ya que es el único que tiene la copia actualizada. Lo enviará en un **RpBloque** y marcará su copia como compartida o inválida, según fuera la petición.
- Cuando un bloque modificado sale de la caché por reemplazo, se genera una petición de postescritura **PtPEsc**.

En la siguiente tabla se refleja el comportamiento de la cache:

Estado inicial	Evento	Acción	Estado siguiente
	PrLec	Acierto	Modificado
	PrEsc	Acierto	Modificado
Modificado	PtLec	RpBloque	Compartido
	PtLecEx	RpBloque	Inválido
	Reemplazo	PtPEsc	Inválido
	PrLec	Acierto	Compartido
Compartido	PrEsc	PtLecEx	Modificado
	PtLec	Nada	Compartido
	PtLecEx	Invalidar	Inválido
	PrLec	PtLec	Compartido
Inválido	PrEsc	PtLecEx	Modificado
	PtLec	Nada	Inválido
	PtLecEx	Nada	Inválido

Existen desventajas al usar MSI, fallos que degradan el rendimiento del protocolo, son los fallos de compartición verdadera y compartición falsa.

Para tratar de reducir las desventajas existen protocolos alternativos:

MESI

Se añade un nuevo estado y se modifican los otros tres:

- **Modificado:** es el único componente del sistema con una copia válida. El resto de cachés y MP tienen una copia inválida.
- **Exclusivo:** es la única caché con una copia válida, junto a la MP. Cuando se escribe a este bloque no hace falta mandar una señal de invalidación.
- **Compartido:** el bloque es valido en esta caché, en MP y en al menos otra caché.
- **Inválido:** el bloque no está en la caché o su copia ha sido invalidada.

El estado exclusivo actúa como una variación del compartido que permite "promocionar" a modificado sin enviar invalidaciones.

Además de MESI existen otras extensiones.

Problemas del snooping

Como ya se ha comentado utilizar snooping genera un alto tráfico en el bus y se necesita un bus común para todos los procesadores. Esto no siempre está disponible, especialmente en sistemas NUMA donde cada procesador accede a la memoria por separado.

Protocolos basados en directorios

Se utilizan cuando el snooping es inviable. Se requiere un directorio de memoria que contiene y actualiza información sobre el estado de compartición de los bloques.

La idea es seguir la pista de cada bloque de caché, que cachés tienen copia y en qué estado está. De cada bloque de MP se almacenan dos datos:

- **Cachés:** N bits indicando en cuales de las N cachés está presente el bloque.
- **Estado:** bits describiendo el estado del bloque:
 - Local: el bloque está actualizado y no está en ninguna caché
 - Compartido: el bloque está en múltiples cachés y la memoria está actualizada
 - Exclusivo: hay una copia del bloque en estado modificado o exclusivo en una caché.

Los protocolos de snooping se basan en difusión. Sin embargo, los de directorios se basan en transmisiones específicas de punto a punto, se aligera el tráfico por el bus.

Directorios centralizados

Un único directorio gestiona el estado de los bloques. Se puede utilizar en sistemas con acceso uniforme a memoria

Directorio distribuido en módulos de MP

En sistemas con acceso a memoria no uniforme, el directorio se distribuye entre los distintos puntos de acceso a memoria (uno por procesador).

Conclusiones

Los protocolos de snooping son más fáciles de implementar pero presentan problemas de escalabilidad. En ese caso, los protocolos basados en directorios son más eficaces.